

Experiment-1

Objective1: To take an array and output the elements with their address.

Code:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    cout<<"Dishank Devrani"<<endl<<"Roll no: 2503201000473"<<endl;
    int n;
    cout<<"Enter size of the array: "<<endl;
    cin>>n;

    int a[n];
    cout<<"Enter the elements for the array: "<<endl;
    for(int i=0;i<n;i++){
        cin>>a[i];
    }

    cout<<endl;

    for(int i=0;i<n;i++){
        cout<<"The element is: "<<a[i]<<" with the address as: "<<&a[i]<<endl;
    }

    return 0;
}
```

Output:

```
Dishank Devrani
Roll no: 2503201000473
Enter size of the array:
5
Enter the elements for the array:
1
2
3
4
5

The element is: 1 with the address as: 0x61feb8
The element is: 2 with the address as: 0x61febc
The element is: 3 with the address as: 0x61fec0
The element is: 4 with the address as: 0x61fec4
The element is: 5 with the address as: 0x61fec8
PS C:\Dishank\C++ codes\All the college work\semester 3\ADS lab> |
```

Objective2: To push and pop an element from a stack.

Code:

```
#include <bits/stdc++.h>
using namespace std;

const int MAX = 10;
int Stack[MAX];
int top = -1;

void Push(int value) {
    if (top >= MAX - 1) {
        cout << "Stack Overflow" << endl;
    }
    else {
        top++;
        Stack[top] = value;
        cout << value << " pushed into stack." << endl;
    }
}
```

```

void Pop() {
    if (top == -1) {
        cout << "Stack is underflow" << endl;
    }
    else {
        cout << "Deleted element " << Stack[top] << endl;
        top = top - 1;
    }
}

void display() {
    if (top == -1) {
        cout << "Stack is empty." << endl;
    }
    else {
        cout << "Stack elements are:" << endl;
        for (int i = top; i >= 0; i--) {
            cout << Stack[i] << endl;
        }
    }
}

```

```

int main() {
    cout<<"Dishank Devrani"<<endl<<"Roll no: 2503201000473"<<endl;

    int n, value;

    cout << "Enter number of elements to push: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Enter element: ";
        cin >> value;
        Push(value);
    }

    cout << "Popping top element..." << endl;
    Pop();

    display();

    return 0;
}

```

Output:

```
Dishank Devrani
Roll no: 2503201000473
Enter number of elements to push: 5
Enter element: 1
1 pushed into stack.
Enter element: 2
2 pushed into stack.
Enter element: 3
3 pushed into stack.
Enter element: 4
4 pushed into stack.
Enter element: 5
5 pushed into stack.
Popping top element...
Deleted element 5
Stack elements are:
4
3
2
1
PS C:\Dishank\C++ codes\All the college work\semester 3\ADS lab>
```

Objective3: To convert Infix to Postfix using Stack.

Code:

```
#include <bits/stdc++.h>
using namespace std;

int precedence(char op){
    if(op=='^')
        return 3;
    else if(op=='*' || op=='/')
        return 2;
    else if(op=='+' || op=='-')
        return 1;
    else
        return 0;
}
```

```

string infixToPostfix(string s){
    stack<char> st;
    string postfix = "";

    for(int i=0; i<s.length(); i++){
        char ch = s[i];

        if(isalnum(ch)){
            postfix += ch;
        }
        else if(ch=='('){
            st.push(ch);
        }
        else if(ch==')'){
            while(!st.empty() && st.top()!='('){
                postfix += st.top();
                st.pop();
            }
            if(!st.empty()){
                st.pop();
            }
        }
        else{
            while(!st.empty() && ((precedence(st.top()) > precedence(ch)) ||
                (precedence(st.top()) == precedence(ch) && ch != '^'))){
                postfix += st.top();
                st.pop();
            }
            st.push(ch);
        }
    }

    while(!st.empty()){
        postfix += st.top();
        st.pop();
    }

    return postfix;
}

```

```

int main(){

    cout<<"Dishank Devrani"<<endl<<"Roll no: 2503201000473"<<endl;

    string s;
    cout << "Enter infix expression: ";
    cin >> s;

    cout << "Postfix expression: " << infixToPostfix(s);

    return 0;
}

```

Output:

```

Dishank Devrani
Roll no: 2503201000473
Enter infix expression: A^B*C+D
Postfix expression: AB^C*D+
PS C:\Dishank\C++ codes\All the college work\semester 3\ADS lab>

```

Objective4: To convert infix to prefix using Stack

Code:

```
#include <bits/stdc++.h>
using namespace std;

int precedence(char op){
    if(op=='^')
        return 3;
    else if(op=='*' || op=='/')
        return 2;
    else if(op=='+' || op=='-')
        return 1;
    else
        return 0;
}
```

```

bool isOperator(char ch){
    return (ch=='+' || ch=='-' || ch=='*' || ch=='/' || ch=='^');
}

string infixToPrefix(string s){
    reverse(s.begin(), s.end());

    for(int i=0; i<s.length(); i++){
        if(s[i]=='(')
            s[i]=')';
        else if(s[i]==')')
            s[i]='(';
    }
}

```

```

stack<char> st;
string prefix = "";

for(int i=0; i<s.length(); i++){
    char ch = s[i];

    if(isalnum(ch)){
        prefix += ch;
    }
    else if(ch=='('){
        st.push(ch);
    }
}

```

```

    }
    else if(ch==' '){
        while(!st.empty() && st.top()!='('){
            prefix += st.top();
            st.pop();
        }
        if(!st.empty())
            st.pop();
    }
    else if(isOperator(ch)){
        while(!st.empty() && ((precedence(st.top()) > precedence(ch)) ||
            (precedence(st.top()) == precedence(ch) && ch == '^'))){
            prefix += st.top();
            st.pop();
        }
        st.push(ch);
    }
}
}

```

```

while(!st.empty()){
    prefix += st.top();
    st.pop();
}

reverse(prefix.begin(), prefix.end());
return prefix;
}

```

```
int main(){  
  
    cout<<"Dishank Devrani"<<endl<<"Roll no: 2503201000473"<<endl;  
  
    string s;  
    cout << "Enter infix expression: ";  
    cin >> s;  
  
    cout << "Prefix expression: " << infixToPrefix(s);  
  
    return 0;  
}
```

Output:

```
Dishank Devrani  
Roll no: 2503201000473  
Enter infix expression: A^B+C*D-E  
Prefix expression: -+^AB*CDE  
PS C:\Dishank\C++ codes\All the college work\semester 3\ADS lab>
```